

DSAA2011 Machine Learning

Midterm review (L1-L6)

Dr. Zixin Zhong

Data Science and Analytics Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

March 9, 2026

Midterm: 20 points/100 points

⦿ How?

- Closed-paper exam
- NO cheat sheet
- NO coding with computer
- ⦿ Permitted calculators: Basic non-programmable models only. Devices must not have graphing, CAS, or programmable features.

⦿ What is covered?

- Everything introduced during lectures 01-06 and labs 01-06
- Lecture notes and lab notes (solution set)

Summary of each lecture session: see the first few slides of the next lecture

⦿ Red line?

- NO cheating!



Memorize everything: formula, equation, proof ...?

Understand the idea of algorithm design
See the beauty and importance of math in ML



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering
- 6 Model Selection + Feature selection



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering
- 6 Model Selection + Feature selection



Recap of Lecture 1

- Definition and taxonomy of machine learning
- Mathematical tools



Taxonomy of Machine Learning (A Simplistic View)

♠ What type of data?

- **Supervised learning** - **labeled data**: e.g., prediction
- **Semi-supervised learning**
- **Unsupervised learning** - **unlabeled data**: e.g., clustering
- **Reinforcement learning** - **environment feedback**: e.g., multi-armed bandit

♠ When do we collect data?

- **Offline learning**
- **Online learning**



Mathematical tools

- Set and function
 - ▶ Set
 - ▶ Function: differentiable, (strictly) convex/concave, linear and affine
 - ▶ Local and global extrema, partial derivative and gradient vector



Mathematical tools

- Set and function
 - ▶ Set
 - ▶ Function: differentiable, (strictly) convex/concave, linear and affine
 - ▶ Local and global extrema, partial derivative and gradient vector
 - ★ $f(x)$ is differentiable and (strictly)convex: x_0 such that $f'(x_0) = 0$ is the (unique) minimizer of function f .
 - ★ $f(x)$ is differentiable and (strictly) concave: x_0 such that $f'(x_0) = 0$ is the (unique) maximizer of function f .



Mathematical tools

- Set and function
 - ▶ Set
 - ▶ Function: differentiable, (strictly) convex/concave, linear and affine
 - ▶ Local and global extrema, partial derivative and gradient vector
- Probability and estimation
 - ▶ Discrete and continuous random variables, event
 - ▶ Sum rule and product rule
 - ▶ Bayes' rule
 - ▶ Independence, expectation and variance
 - ▶ **Maximum likelihood estimation (MLE) (unformal):**
Find the parameter set that maximizes the probability the given dataset is collected
 - ▶ Exponential distribution: memory-less



Bayes' rule

$$p(y|x) = \frac{p(x|y)p(y)}{\sum_{y' \in \mathcal{Y}} p(x|y')p(y')} \propto p(x|y)p(y) \text{ (when } x \text{ is fixed)}$$

- **Prior:** beliefs or knowledge about y before observing the data x
- **Likelihood:** how likely the observed x is, given a particular value of y
- **Posterior:** updated belief about y after incorporating the prior and the likelihood of the observed data



Function of random variable

If g is a function from the domain of X to $\mathbb{R}$, we can obtain the expectation of $Y = g(X)$ in the same way.

$$\text{(Discrete case)} \quad \mathbb{E}Y = \mathbb{E}[g(X)] = \sum_y yp_Y(y) = \sum_{i=1}^m g(x)p_X(x),$$

$$\text{(Continuous case)} \quad \mathbb{E}Y = \mathbb{E}[g(X)] = \int_{\mathbb{R}} yf_Y(y)dy = \int_{\mathbb{R}} g(x)f_X(x)dx.$$



Maximum likelihood estimation (MLE)

$$\begin{aligned}\hat{\theta}_{\text{ML}} &= \arg \max_{\theta \in (0,1)} \underbrace{p_{X_1, X_2, \dots, X_m}(X_1, X_2, \dots, X_m; \theta)}_{\text{Likelihood}} \\ &= \arg \max_{\theta \in (0,1)} \underbrace{\log p_{X_1, X_2, \dots, X_m}(X_1, X_2, \dots, X_m; \theta)}_{\text{Log likelihood}} \\ &= \arg \max_{\theta \in (0,1)} \prod_{i=1}^m p_X(X_i; \theta) \quad (\text{when } X_i\text{'s are independent})\end{aligned}$$

- **Likelihood**: probability of observing the dataset $\mathcal{D}$ given a set of parameters θ
- **Maximum**: seeks the set of parameters θ that maximizes this likelihood function, i.e., makes the observed dataset as “likely” as possible under the model
- Concept of MLE: linear regression, logistic regression, ...



Maximum likelihood estimation (MLE)

- **Consistency.** as the sample size increases to infinity, the estimator will converge to the true parameter value: $\hat{\theta}_{\text{ML}} \xrightarrow{p} \theta$ as $n \rightarrow \infty$.

- ▶ We say $X_n \xrightarrow{p} X$ as $n \rightarrow \infty$ if

$$\lim_{n \rightarrow \infty} \Pr(|X_n - X| > \varepsilon) = 0 \quad \forall \varepsilon > 0.$$

- **Unbiasedness.** MLE is not necessarily unbiased, but in certain cases, it can be unbiased.
 - ▶ We say an estimator of a given parameter is unbiased if its expected value is equal to the true value of the parameter.



Maximum likelihood estimation (MLE)

- **Efficiency.** MLE is asymptotically efficient in large samples, i.e., it achieves the lowest possible variance among all unbiased estimators.
 - ▶ Cramér-Rao Lower Bound (CRLB): theoretical lower bound for the variance of any unbiased estimator.

- **Asymptotic Normality.** MLE is asymptotically normal:

$$\sqrt{n}(\hat{\theta}_{\text{ML}} - \theta) \xrightarrow{d} \mathcal{N}(0, I(\theta)^{-1}).$$

- ▶ $I(\theta)$: Fisher information of θ (larger information $\implies$ smaller variance).
- ▶ We say $X_n \xrightarrow{d} X$ as $n \rightarrow \infty$ if

$$\lim_{n \rightarrow \infty} F_{X_n}(x) = F_X(x) \quad \forall x \text{ at which } F_X(x) \text{ is continuous.}$$

- **Model Sensitivity.** MLE is sensitive to model assumptions, and incorrect assumptions can lead to biased or inconsistent estimates.



Maximum likelihood estimation (MLE)

- **Consistency.** as the sample size increases to infinity, the estimator will converge to the true parameter value: $\hat{\theta}_{\text{ML}} \xrightarrow{P} \theta$ as $n \rightarrow \infty$.
- **Unbiasedness.** MLE is not necessarily unbiased, but in certain cases, it can be unbiased.
- **Efficiency.** MLE is asymptotically efficient in large samples, i.e., it achieves the lowest possible variance among all unbiased estimators.
- **Asymptotic Normality.** MLE is asymptotically normal:
$$\sqrt{n}(\hat{\theta}_{\text{ML}} - \theta) \xrightarrow{d} \mathcal{N}(0, I(\theta)^{-1}).$$
- **Model Sensitivity.** MLE is sensitive to model assumptions, and incorrect assumptions can lead to biased or inconsistent estimates.



Mathematical tools

- Set and function
 - ▶ Set
 - ▶ Function: differentiable, (strictly) convex/concave, linear and affine
 - ▶ Local and global extrema, partial derivative and gradient vector
- Probability and estimation
 - ▶ Discrete and continuous random variables, event
 - ▶ Sum rule and product rule
 - ▶ Bayes' rule
 - ▶ Independence, expectation and variance
 - ▶ **Maximum likelihood estimation (MLE) (unformal):**
Find the parameter set that maximizes the probability the given dataset is collected
 - ▶ Exponential distribution: memory-less
- Systems of linear equations
 - ▶ Vector: linear independence
 - ▶ Matrix: rank, kernel, range, dimension, invertible
 - ▶ Solution to linear system: existence and uniqueness, Gaussian-elimination method



Solutions to linear systems

◎ **Method:** apply **Guassian-elimination** method to the augmented matrix $\tilde{\mathbf{X}} = [\mathbf{X} \ \mathbf{y}]$.

$$\mathbf{X}\mathbf{w} = \mathbf{y} \text{ or } [\underline{x}_1 \ \underline{x}_2 \ \dots \ \underline{x}_d] \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_d \end{bmatrix}, \text{ where } \underline{x}_i = \begin{bmatrix} x_{1,i} \\ x_{2,i} \\ \vdots \\ x_{m,i} \end{bmatrix}. \quad (1.1)$$

Theorem 1.1 (Rouché-Capelli Theorem)

- The system in (1.1) admits a unique solution if and only if $\text{rank}(\mathbf{X}) = \text{rank}(\tilde{\mathbf{X}}) = d$;
- The system in (1.1) has no solution if and only if $\text{rank}(\mathbf{X}) < \text{rank}(\tilde{\mathbf{X}})$;
- The system in (1.1) has infinitely many solutions if and only if $\text{rank}(\mathbf{X}) = \text{rank}(\tilde{\mathbf{X}}) < d$.



Recommended materials for mathematical tools

- Set and function
 - ▶ ‘The Matrix Cookbook’ [<http://matrixcookbook.com>] by Kaare Brandt Petersen and Michael Syskind Pedersen
- Probability and estimation
 - ▶ ‘A First Course in Probability’ by Sheldon Ross
 - ▶ ‘Probability and Statistics’ by Morris H. DeGroot and Mark J. Schervish
 - ▶ ‘Probability Theory’ by Achim Klenke
- Linear algebra:
 - ▶ ‘Introduction to Linear Algebra’ by Gilbert Strang
(<https://math.mit.edu/~gs/linearalgebra/ila5/indexila5.html>)
- Found in Canvas (‘Modules’ > ‘Recommended materials’) or online



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering
- 6 Model Selection + Feature selection



Recap of Lecture 2

- Least squares and linear regression
 - Linear regression with multiple outputs
 - Linear regression and MLE
- Linear classification
 - Linear models for binary classification
 - Linear models for multi-class classification
- Polynomial regression/classification



Linear regression

- **Learning/Training:** Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, the least squares solution (with offset) is

$$\bar{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}$$

where the **design matrix** and **target vector** are

$$\mathbf{X} = \begin{bmatrix} -\bar{\mathbf{x}}_1^\top & - \\ -\bar{\mathbf{x}}_2^\top & - \\ \vdots & \\ -\bar{\mathbf{x}}_m^\top & - \end{bmatrix} = \begin{bmatrix} 1 & -\mathbf{x}_1^\top & - \\ 1 & -\mathbf{x}_2^\top & - \\ \vdots & \vdots & \\ 1 & -\mathbf{x}_m^\top & - \end{bmatrix} \in \mathbb{R}^{m \times (d+1)} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \in \mathbb{R}^m.$$

- **Prediction/Testing:** Given a new feature vector (sample, example) $\mathbf{x}_{\text{new}}$, the prediction based on the least squares solution is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \bar{\mathbf{w}}^* = b^* + \mathbf{x}_{\text{new}}^\top \mathbf{w}^*.$$



Linear regression with multiple outputs

- Suppose there are h outputs we want to predict (above $h = 1$).
- Given a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ (column vector) and $\mathbf{y}_i \in \mathbb{R}^{1 \times h}$ (row vector), the model to be used is

$$\underbrace{\begin{bmatrix} y_{1,1} & y_{1,2} & \dots & y_{1,h} \\ y_{2,1} & y_{2,2} & \dots & y_{2,h} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m,1} & y_{m,2} & \dots & y_{m,h} \end{bmatrix}}_{\mathbf{Y} \in \mathbb{R}^{m \times h}} = \underbrace{\begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ 1 & x_{2,1} & \dots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \dots & x_{m,d} \end{bmatrix}}_{\mathbf{X} \in \mathbb{R}^{m \times (d+1)}} \underbrace{\begin{bmatrix} b_1 & b_2 & \dots & b_h \\ w_{1,1} & w_{1,2} & \dots & w_{1,h} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \dots & w_{d,h} \end{bmatrix}}_{\bar{\mathbf{W}} \in \mathbb{R}^{(d+1) \times h}}$$

- When $h = 1$, this particularizes to standard linear regression.
- This is exactly h separate linear regression problems.



Linear regression with multiple outputs

- Suppose there are h outputs we want to predict (above $h = 1$).
- Given a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ (column vector) and $\mathbf{y}_i \in \mathbb{R}^{1 \times h}$ (row vector), the model to be used is

$$\underbrace{\begin{bmatrix} y_{1,1} & y_{1,2} & \dots & y_{1,h} \\ y_{2,1} & y_{2,2} & \dots & y_{2,h} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m,1} & y_{m,2} & \dots & y_{m,h} \end{bmatrix}}_{\mathbf{Y} \in \mathbb{R}^{m \times h}} = \underbrace{\begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ 1 & x_{2,1} & \dots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \dots & x_{m,d} \end{bmatrix}}_{\mathbf{X} \in \mathbb{R}^{m \times (d+1)}} \underbrace{\begin{bmatrix} b_1 & b_2 & \dots & b_h \\ w_{1,1} & w_{1,2} & \dots & w_{1,h} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \dots & w_{d,h} \end{bmatrix}}_{\overline{\mathbf{W}} \in \mathbb{R}^{(d+1) \times h}}$$

- When $h = 1$, this particularizes to standard linear regression.
- This is exactly h separate linear regression problems.



Linear regression with multiple outputs

- **Learning/Training:** Least Squares Solution

$$\overline{\mathbf{W}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} \in \mathbb{R}^{(d+1) \times h}.$$

- **Prediction/Testing:** Given a new feature vector $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, we can predict its h outputs as

$$\hat{\mathbf{y}}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^* \in \mathbb{R}^{1 \times h}$$

- The k -th ($1 \leq k \leq h$) component of $\hat{\mathbf{y}}_{\text{new}}$ is the prediction of the k -th output based the dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$.

⊙ Is the matrix $\mathbf{X}^\top \mathbf{X}$ invertible?



MLE and linear regression

- Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + b + e_i$ for each data point i and error $e_i \sim \mathcal{N}(0, \sigma^2)$.
- Likelihood function for the entire dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ is

$$L(\mathbf{W}, \sigma^2 \mid \{y_i, \mathbf{x}_i\}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{W}^\top \mathbf{x}_i)^2}{2\sigma^2}\right)$$

- If $\mathbf{X}$ has full column rank, $\mathbf{X}^\top \mathbf{X}$ is invertible and the maximizer $(\hat{\mathbf{w}}, \hat{\sigma}^2)$ is:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (\text{least squares solution}),$$

$$\hat{\sigma}^2 = \frac{1}{m} \sum_{i=1}^m (y_i - [1 \ x_i^\top] \cdot \hat{\mathbf{w}})^2 = \frac{1}{m} (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}).$$

- ◉ **MLE of distribution of error:** $e_i \sim \mathcal{N}(0, \hat{\sigma}^2) = \mathcal{N}(0, (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})/m)$.



MLE and linear regression

- If $\mathbf{X} = [\bar{\mathbf{x}}_1^\top \ \bar{\mathbf{x}}_2^\top \ \dots \ \bar{\mathbf{x}}_n^\top]^\top$ has full column rank, $\mathbf{X}^\top \mathbf{X}$ is invertible and

$$\frac{\partial}{\partial \bar{\mathbf{w}}} \log L(\bar{\mathbf{w}}, \sigma^2 \mid \{y_i, \mathbf{x}_i\}) = \frac{1}{\sigma^2} \sum_{i=1}^m (y_i - \bar{\mathbf{w}}^\top \bar{\mathbf{x}}_i) \bar{\mathbf{x}}_i = \mathbf{0}_{(d+1) \times 1} \quad (2.1)$$

$$\Rightarrow \bar{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Proof. Firstly, (2.1) can be rewritten as $\sum_{i=1}^m (\bar{\mathbf{w}}^\top \bar{\mathbf{x}}_i) \bar{\mathbf{x}}_i = \sum_{i=1}^m y_i \bar{\mathbf{x}}_i$.



MLE and linear regression

Proof. Firstly, (2.1) can be rewritten as $\sum_{i=1}^m (\bar{\mathbf{w}}^\top \bar{\mathbf{x}}_i) \bar{\mathbf{x}}_i = \sum_{i=1}^m y_i \bar{\mathbf{x}}_i$. Then observe that

$$\sum_{i=1}^m y_i \bar{\mathbf{x}}_i = [\bar{\mathbf{x}}_1 \quad \bar{\mathbf{x}}_2 \quad \dots \quad \bar{\mathbf{x}}_n] \cdot \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \mathbf{X}^\top \mathbf{y},$$

$$\begin{aligned} \sum_{i=1}^m (\bar{\mathbf{w}}^\top \bar{\mathbf{x}}_i) \bar{\mathbf{x}}_i &= \sum_{i=1}^m (\bar{\mathbf{x}}_i^\top \bar{\mathbf{w}}) \bar{\mathbf{x}}_i = \sum_{i=1}^m \bar{\mathbf{x}}_i (\bar{\mathbf{x}}_i^\top \bar{\mathbf{w}}) = \sum_{i=1}^m (\bar{\mathbf{x}}_i \bar{\mathbf{x}}_i^\top) \bar{\mathbf{w}} = \left(\sum_{i=1}^m \bar{\mathbf{x}}_i \bar{\mathbf{x}}_i^\top \right) \bar{\mathbf{w}} \\ &= [\bar{\mathbf{x}}_1 \quad \bar{\mathbf{x}}_2 \quad \dots \quad \bar{\mathbf{x}}_m] \cdot \begin{bmatrix} \bar{\mathbf{x}}_1^\top \\ \bar{\mathbf{x}}_2^\top \\ \vdots \\ \bar{\mathbf{x}}_m^\top \end{bmatrix} \cdot \bar{\mathbf{w}} = \mathbf{X}^\top \mathbf{X} \bar{\mathbf{w}}. \end{aligned}$$



Then (2.1) can be further rewritten as $\mathbf{X}^\top \mathbf{X} \bar{\mathbf{w}} = \mathbf{X}^\top \mathbf{y}$.

MLE and linear regression: why Gaussian noise?

Theorem 2.1 (Central limit theorem (CLT))

Suppose $X_1, X_2, X_3 \dots$ is a sequence of i.i.d. random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}[X_i] = \sigma^2 < \infty$. Then, as $n \rightarrow \infty$, the distribution of $\sqrt{n}(\bar{X}_n - \mu)$ converges to $\mathcal{N}(0, \sigma^2)$:

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$$

- We say $X_n \xrightarrow{d} X$ as $n \rightarrow \infty$ if

$$\lim_{n \rightarrow \infty} F_{X_n}(x) = F_X(x) \quad \forall x \text{ at which } F_X(x) \text{ is continuous.}$$

- In the case $\sigma > 0$, CLT implies that the cumulative distribution functions (cdf) of $\sqrt{n}(\bar{X}_n - \mu)$ converge pointwise to the cdf of the $\mathcal{N}(0, \sigma^2)$ distribution:

$$\lim_{n \rightarrow \infty} \mathbb{P}[\sqrt{n}(\bar{X}_n - \mu) \leq z] = \lim_{n \rightarrow \infty} \mathbb{P}\left[\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \leq \frac{z}{\sigma}\right] = \Phi\left(\frac{z}{\sigma}\right)$$

 where $\Phi(z)$ is the standard normal cdf evaluated at z .

Linear models for binary classification

- ♠ **Main idea:** to treat binary classification as regression where each label y_i can only take on -1 or $+1$.
- **Learning/Training:** given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ (where each $y_i \in \{+1, -1\}$), learn the weights using least squares

$$\bar{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}.$$

- **Prediction/Testing:** given a new data sample $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, its predicted label is

$$\hat{y}_{\text{new}} = \text{sign} \left(\bar{\mathbf{x}}_{\text{new}}^\top \bar{\mathbf{w}}^* \right) = \text{sign} \left(\begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \bar{\mathbf{w}}^* \right) \in \{+1, -1\}.$$



Linear models for multi-class classification

- (Training/Learning) The design matrix $\mathbf{X}$ is the same. If it has full column rank, find the least squares solution

$$\overline{\mathbf{W}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} \in \mathbb{R}^{(d+1) \times C}.$$

- (Testing/Prediction) Given a new feature vector $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, we have

$$\hat{y}^{\text{new,reg}}[:, k] = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^*[:, k] \quad \forall k \in \{1, 2, \dots, C\}$$

and predict its class as

$$\hat{y}_{\text{new}} = \underset{k \in \{1, 2, \dots, C\}}{\text{arg max}} \left(\begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^*[:, k] \right) \in \{1, 2, \dots, C\}$$

where $\overline{\mathbf{W}}^*[:, k] \in \mathbb{R}^{d+1}$ is the k -column of $\overline{\mathbf{W}}^*$.



Polynomial regression/classification

- **Learning/Training:**

$$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top)^{-1} \mathbf{y}$$

where

$$\mathbf{P} = \mathbf{P}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) = \begin{bmatrix} -\mathbf{p}_1^\top & - \\ -\mathbf{p}_2^\top & - \\ \vdots & \\ -\mathbf{p}_m^\top & - \end{bmatrix} \in \mathbb{R}^{m \times \binom{d+p}{p}}$$

- **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*.$$



Polynomial regression/classification

- For regression applications:

- ▶ Learn continuous-valued y by using either primal or dual forms
- ▶ Prediction:

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^{\top} \mathbf{w}^*.$$

- For classification applications:

- ▶ Learn **discrete-valued** $y \in \{-1, +1\}$ (for binary classification) or **one-hot encoded** $\mathbf{Y}$ (for $y \in \{1, 2, \dots, C\}$ for multi-class classification) using either primal or dual forms
- ▶ **Binary prediction**

$$\hat{y}_{\text{new}} = \text{sign} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{w}^* \right)$$

- ▶ **Multi-class prediction**

$$\hat{y}_{\text{new}} = \arg \max_{k \in \{1, 2, \dots, C\}} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{W}^*[:, k] \right)$$



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)**
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering
- 6 Model Selection + Feature selection



Recap of Lecture 3

- Ridge regression
- Gradient descent



Ridge regression (linear form)

- Ridge regression in primal form (when $m > d$)

- ▶ **Learning/Training:**

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}$$

- ▶ **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \mathbf{w}^*$$

where $\mathbf{p}_{\text{new}}$ is the polynomial vector associated to $\mathbf{x}_{\text{new}}$.

- Ridge regression in dual form (when $m < d' = \binom{p+d}{p}$)

- ▶ **Learning/Training:**

$$\mathbf{w}^* = \mathbf{X}^\top (\mathbf{X} \mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}$$

- ▶ **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \mathbf{w}^*$$



Ridge regression (linear form)

- **Fact 1:** For every $\lambda > 0$, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$ is invertible.
- **Hint:** $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$ is positive-definite.

- **Fact 2:** For every $\lambda > 0$,

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{X} \mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}. \quad (\text{P-D})$$

- **Hint:** To show (P-D), we use the **Woodbury formula**

$$(\mathbf{I} + \mathbf{U} \mathbf{V})^{-1} = \mathbf{I} - \mathbf{U} (\mathbf{I} + \mathbf{V} \mathbf{U})^{-1} \mathbf{V}.$$



Review of ridge regression (polynomial form)

- Ridge regression in primal form (when $m > d' = \binom{p+d}{p}$)

- ▶ **Learning/Training:**

$$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}$$

- ▶ **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$

where $\mathbf{p}_{\text{new}}$ is the polynomial vector associated to $\mathbf{x}_{\text{new}}$.

- Ridge regression in dual form (when $m < d' = \binom{p+d}{p}$)

- ▶ **Learning/Training:**

$$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P} \mathbf{P}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$

- ▶ **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$



Regression/classification

- For regression applications:

- ▶ Learn continuous-valued y by using either primal or dual forms
- ▶ Prediction:

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^{\top} \mathbf{w}^*.$$

- For classification applications:

- ▶ Learn **discrete-valued** $y \in \{-1, +1\}$ (for binary classification) or **one-hot encoded** $\mathbf{Y}$ (for $y \in \{1, 2, \dots, C\}$ for multi-class classification) using either primal or dual forms
- ▶ **Binary prediction**

$$\hat{y}_{\text{new}} = \text{sign} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{w}^* \right)$$

- ▶ **Multi-class prediction**

$$\hat{y}_{\text{new}} = \arg \max_{k \in \{1, 2, \dots, C\}} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{W}^*[:, k] \right)$$



Useful Python packages and functions

- `sklearn.preprocessing PolynomialFeatures`,
- `np.sign`,
- `sklearn.model_selection train_test_split`,
- `sklearn.preprocessing OneHotEncoder`



[Exercise] $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ ($\forall \lambda \leq 0$) is always invertible?

- We wish to solve

$$\bar{\mathbf{w}}^* = \arg \min_{\bar{\mathbf{w}}=[b, \mathbf{w}]^\top} (\mathbf{X}\bar{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\bar{\mathbf{w}} - \mathbf{y}) + \lambda \bar{\mathbf{w}}^\top \bar{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$(\mathbf{X}\bar{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\bar{\mathbf{w}} - \mathbf{y}) + \lambda \bar{\mathbf{w}}^\top \bar{\mathbf{w}} = \bar{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \bar{\mathbf{w}} - 2\bar{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y}$$

- Differentiating w.r.t. $\bar{\mathbf{w}}$ and setting the result to zero yields

$$2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \bar{\mathbf{w}}^* = 2(\mathbf{X}^\top \mathbf{y}) \iff \bar{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- For any $\lambda \leq 0$, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is always invertible?

- For any $\lambda < 0$, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is always invertible?

- Counter example?



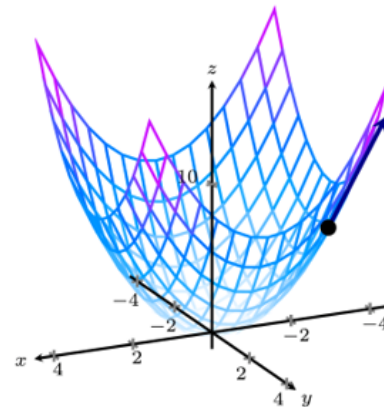
[Exercise] $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ ($\forall \lambda \leq 0$) is always invertible?



Gradient descent (GD) algorithm: motivation

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \dots, w_d]^\top$

- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



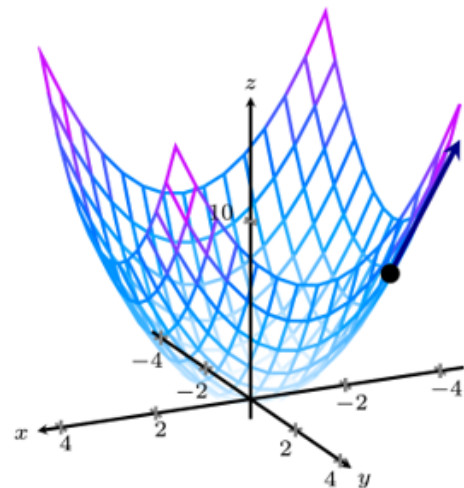
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$ (hard to find the solution of $\nabla_{\mathbf{w}} C(\mathbf{w})$)
- $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is **increasing** most rapidly
 - $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is **decreasing** most rapidly
- ◉ Hint: Think about the case with $\mathbf{w} \in \mathbb{R}^1$.
- ◉ Book: 'Multivariable Calculus' by James Stewart



Gradient descent (GD) algorithm: basics

Algorithm 1: Gradient Descent (GD)

- 1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
 - 2: **while** true **do**
 - 3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
 - 4: **if** converge **then**
 - 5: **return** $\mathbf{w}_{k+1}$
 - 6: **end if**
 - 7: **end while**
-



- According to multi-variable calculus, if η is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{w}_k)$.
 $\Rightarrow$ We get better $\mathbf{w}$ after each iteration.



Gradient descent (GD) algorithm: basics

⊙ Possible convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold
- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold

⊙ Limitation: Gradient descent can only find local minimum

- Because gradient = $\mathbf{0}$ at local minimum
so $\mathbf{w}$ won't change after that

⊙ Impact of η

- η is too small:
 - ▶ Converge slowly
- η is too large:
 - ▶ Overshoot the local
 - ▶ Converge slowly to the minimum or may never converge



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in **objective function C** below a threshold
 - ▶ Pro: direct measure of optimization progress, prevents unnecessary iterations
 - ▶ Con: computational cost, sensitive to scale, NOT guarantee convergence of $\mathbf{w}$
- Check percentage or absolute change in **parameter $\mathbf{w}$** below a threshold
 - ▶ Pro: stable solution, less sensitive to oscillations in the function value
 - ▶ Con: can take longer to detect convergence, NOT directly reflect progress in objective function

⊙ Change of function C or parameter $\mathbf{w}$?

- Function C : care primarily about minimizing C , C is well-behaved (e.g., smooth and convex), low computational cost
- Parameter $\mathbf{w}$: care primarily if the parameters are stable and the optimization process has converged

○ Do hybrid?



Variation/convergence of gradient descent (GD)

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:
Decreasing learning rate, Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate
- Different 'gradient':
Momentum-based GD, Nesterov Accelerated Gradient (NAG)
- Design of cost function:
Batched GD, Stochastic GD, Mini-Batch GD
- Adam (Adaptive Moment Estimation), Newton
- ...
- Book: 'Convex Optimization' by Stephen Boyd and Lieven Vandenberghe
Available at <https://stanford.edu/~boyd/cvxbook/>



Variation of gradient descent (GD): changing learning rate

⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

○ Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1/2} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

• $M_k = \text{diag}(\sum_{s=0}^k \nabla_{\mathbf{w}} C(\mathbf{w}_s) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_s))$ - a diagonal matrix where each diagonal entry is the sum of the element-wise squared gradients (i.e., $\nabla_{\mathbf{w}} C(\mathbf{w}_k) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_k)$)

$$[a_1, a_2, a_3]^\top \odot [b_1, b_2, b_3]^\top = [a_1 b_1, a_2 b_2, a_3 b_3]^\top$$

ε : small constant to prevent division by zero

• Element-wise view

$$[\mathbf{w}_{k+1}]_i \leftarrow [\mathbf{w}_k]_i - \frac{\eta}{\sqrt{M_{ii,k} + \varepsilon}} [\nabla_{\mathbf{w}} C(\mathbf{w}_k)]_i, \quad \text{where } M_{ii,k} = \sum_{s=0}^k [\nabla_{\mathbf{w}} C(\mathbf{w}_s)]_i^2$$



Variation of gradient descent (GD): changing learning rate

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

- Changing learning rate:

- 2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1/2} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- **Pros:** adjust the learning rate for based on the historical gradient information
 $\Rightarrow$ gives larger update to $\mathbf{w}_k$ with smaller gradient and smaller update to $\mathbf{w}_k$ with larger gradient.
- **Cons:** learning rate tends to shrink too quickly, leading to slow convergence or getting stuck at suboptimal points (vanishing learning rate).



Variation of gradient descent (GD): different 'gradient'

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

- Different 'gradient':

1. Momentum-based GD:

$$v_k \leftarrow \beta v_{k-1} + (1 - \beta) \nabla_{\mathbf{w}} C(\mathbf{w}_k), \quad \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta v_k$$

β - momentum factor, $v_0 = \nabla_{\mathbf{w}} C(\mathbf{w}_0)$.

Why: maintaining a moving average of past gradients; converging fast but can overshoot the minimum



Variation of gradient descent (GD): different ‘gradient’

⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

○ Different ‘gradient’:

2. Nesterov Accelerated Gradient (NAG):

$$\mathbf{v}_k \leftarrow \beta \mathbf{v}_{k-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta \mathbf{v}_{k-1}), \quad \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \mathbf{v}_k$$

‘**Look ahead**’! - anticipate the direction the optimizer will go in the next step



Variation of gradient descent (GD): different ‘gradient’

⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

○ Different ‘gradient’:

2. Nesterov Accelerated Gradient (NAG):

$$\mathbf{v}_k \leftarrow \beta \mathbf{v}_{k-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta \mathbf{v}_{k-1}), \quad \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \mathbf{v}_k$$

‘**Look ahead**’! - anticipate the direction the optimizer will go in the next step

- **Pros:** faster convergence but slightly more complex
- **Cons:** slightly more complex than momentum alone



Variation of gradient descent (GD): view of dataset

⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

○ View of dataset:

1. Batch Gradient Descent (BGD):

Loss function built using the **entire dataset** at each iteration.

- **Pros:** converges smoothly to the minimum
- **Cons:** computationally expensive, memory intensive, not ideal for online learning (hard to update the model incrementally as new data comes)

Algorithm 2: Batch Gradient Descent (BGD)

```
1: Initialize  $\mathbf{w}_0$  and learning rate/step  
   size  $\eta > 0$   
2: while true do  
3:   Compute  
      $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$   
4:   if converge then  
5:     return  $\mathbf{w}_{k+1}$   
6:   end if  
7: end while
```



Variation of gradient descent (GD): view of dataset

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
 - View of dataset:
- 2. Stochastic Gradient Descent (SGD):
Loss function built using **one randomly chosen sample** at each iteration.
 - Pros: fast updates, suitable for online learning
 - Cons: noisy update, longer convergence time

Algorithm 3: Stochastic GD (SGD)

- 1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
 - 2: **while** true **do**
 - 3: Randomly choose one sample m
 - 4: Build the loss function $C_m(\mathbf{w})$ with the chosen sample m
 - 5: Compute
 $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_m(\mathbf{w}_k)$
 - 6: **if** converge **then**
 - 7: **return** $\mathbf{w}_{k+1}$
 - 8: **end if**
 - 9: **end while**
-



Variation of gradient descent (GD): view of dataset

⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

- View of dataset:

3. Mini-Batch Gradient Descent (MBGD):

Loss function built using **one randomly chosen mini-batch** at each iteration.

- Pros: between SGD and BGD
- Cons: still has noise, hyperparameter tuning (choice of batch size)

Algorithm 4: Mini-Batch GD (MBGD)

- 1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
- 2: Divide the entire dataset into small mini-batches
- 3: **while** true **do**
- 4: Randomly choose one small mini-batch n
- 5: Build the loss function $C_n(\mathbf{w})$ with the **chosen mini-batch** n
- 6: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_n(\mathbf{w}_k)$
- 7: **if** converge **then**
- 8: **return** $\mathbf{w}_{k+1}$
- 9: **end if**
- 10: **end while**



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)**
- 5 Feature engineering
- 6 Model Selection + Feature selection



Recap of Lecture 4

- Logistic regression
- SVM
 - Linearly separable, γ -linearly separable
 - Margin, minimum margin, geometric margin
 - SVM: maximum margin linear classifier, unique solution
 - SVM: offset, slack, leave-one-out cross-validation error



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g(y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0)) = \frac{1}{1 + \exp(-y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))},$$



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g(y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0)) = \frac{1}{1 + \exp(-y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))}, \quad (4.1)$$

- Take the gradients of

$$\log [1 + \exp(-y_t (\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))]$$

with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t [1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)] \quad (4.2)$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t [1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)]. \quad (4.3)$$

- Stochastic gradient descent: update $\boldsymbol{\theta}$ and θ_0 until the termination condition is met.



Summary of logistic regression: regularization

- ⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^n \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^n \frac{1}{1 + \exp(-y_t(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
 - ▶ If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t . By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2 -norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^n \log [1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))]. \quad (4.4)$$



Understand θ^* and θ_0^* (optional)

⊙ Vector of **optimal** prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\theta}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is **no further linearly available information** in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to **improve** the prediction probabilities (or mistake probabilities).

Theorem 4.1

At the optimal solution for the parameter and offset vector θ^, θ_0^* , the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\theta}, \bar{\theta}_0$,*

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \text{where} \quad \mathbf{z} := (\langle \bar{\theta}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \quad (4.5)$$

*Furthermore if a prediction vector $\mathbf{e}$ satisfies (4.5), it has the **minimal ℓ_2 norm** of $(\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta, \theta_0))_{t=1}^n$ over all θ, θ_0 .*

Hint for proof: Assume $\langle \mathbf{z}, \mathbf{e} \rangle \neq 0$, we can prove $\mathbf{e}' := \mathbf{e} - \frac{\langle \mathbf{z}, \mathbf{e} \rangle}{\langle \mathbf{z}, \mathbf{z} \rangle} \mathbf{z}$ satisfies $\|\mathbf{e}'\| < \|\mathbf{e}\|$.



Summary of logistic regression: multi-class classification

- ⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp(-(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0))} \quad (4.6)$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^M) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^M \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \quad (4.7)$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

- Without loss of generality, we can assume that **one of the classes** (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (4.6) is a special case of (4.7).
 - **Why?**



Summary of logistic regression: further exploration

- See the impact of regularization parameter λ
- Try it for multi-class classification
- ◉ Find some dataset from Kaggle:
<https://www.kaggle.com/code/rtatman/datasets-for-regression-analysis>
- ◉ Useful Python packages and functions
`sklearn.linear_model.LogisticRegression`
`numpy.random`
`matplotlib.pyplot.scatter`



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_\theta = \text{sign}(\theta^\top \mathbf{x})$ means

$$\begin{cases} \theta^\top \mathbf{x} > 0, y = +1 \\ \theta^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow y = \text{sign}(\theta^\top \mathbf{x}) \Leftrightarrow y\theta^\top \mathbf{x} > 0.$$

Definition 4.2 (Margin of classifier)

The **margin** of classifier $f_\theta = \text{sign}(\theta^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\theta^\top \mathbf{x} \text{ or } y\langle \theta, \mathbf{x} \rangle$$

- **Positive** margin: $(\mathbf{x}, y)$ is **correctly** classified by θ .
- **Negative** margin: $(\mathbf{x}, y)$ is **not correctly** classified by θ .
- **Bigger** margin: $(\mathbf{x}, y)$ is **more correctly** classified by θ .



Linearly separable

Definition 4.3 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \dots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

$$y_t = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \quad \forall t = 1, 2, \dots, n. \quad (4.8)$$

There is some hyperplane that (**strictly**) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$



γ -linearly separable

Definition 4.4 (γ -linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \dots, n\}$ is **γ -linearly separable** for some $\gamma > 0$ if there exists some $\boldsymbol{\theta}$ such that

$$y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle \geq \gamma \quad \forall t = 1, 2, \dots, n. \quad (4.9)$$

There is some hyperplane that (**strictly**) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$

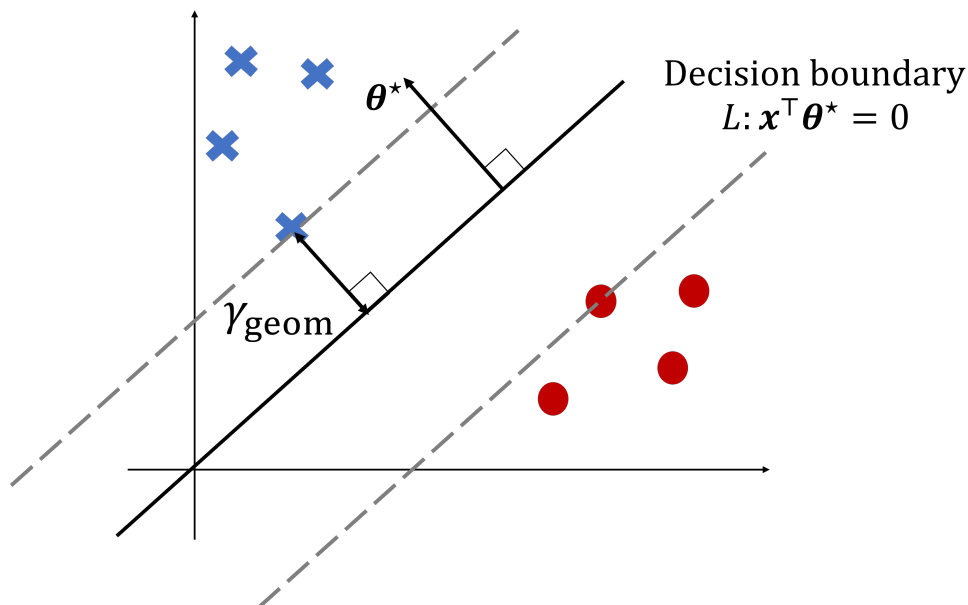


Geometric margin

- Geometric margin:

$$\gamma_{\text{geom}} := \frac{\gamma}{\|\boldsymbol{\theta}\|} = \frac{\min_{1 \leq t \leq n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}.$$

- Smallest distance over all samples to the decision boundary $L = \{x \in \mathbb{R}^d : \langle \mathbf{x}, \boldsymbol{\theta} \rangle = 0\}$.
- $\gamma_{\text{geom}}^{-1}$: fair measure of the difficulty of the classification problem



Support Vector Machine (SVM)

- Maximize the geometric margin:

$$\max_{\boldsymbol{\theta}} \gamma_{\text{geom}} \text{ i.e. } \max_{\boldsymbol{\theta}} \frac{\min_{1 \leq t \leq n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|} \quad (4.10)$$



- PRIMAL-SVM:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \dots, n \quad (4.11)$$



Support Vector Machine (SVM)

- The solution to PRIMAL-SVM is **unique**:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 \quad \text{s. t.} \quad y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \dots, n$$

- **SVM algorithm**: find the maximum (geometric) margin linear classifier
- Why do we seek the maximum (geometric) margin linear classifier?
- Prove the solution to PRIMAL-SVM is **unique**?



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors θ_1, θ_2 ($\theta_1 \neq \theta_2$)
- Then $\|\theta_1\| = \|\theta_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\theta} = (\theta_1 + \theta_2)/2$
- Check feasibility:

Since $\forall t = 1, 2, \dots, n$, $y_t \mathbf{x}_t^\top \theta_1 \geq 1$ and $y_t \mathbf{x}_t^\top \theta_2 \geq 1$, by linearity of inner product, we have

$$y_t \mathbf{x}_t^\top \bar{\theta} = \frac{1}{2} y_t \mathbf{x}_t^\top \theta_1 + \frac{1}{2} y_t \mathbf{x}_t^\top \theta_2 \geq 1$$

$\Rightarrow \bar{\theta}$ is feasible



Primal-SVM: unique solution

Proof (continued):

$$\begin{aligned}\|\bar{\theta}\| &= \left\| \frac{1}{2} (\theta_1 + \theta_2) \right\| && \text{Triangle inequality} && \leq && \frac{1}{2} \|\theta_1\| + \frac{1}{2} \|\theta_2\| \\ & && && = && \|\theta_1\| = \|\theta_2\|\end{aligned}$$

Two case:

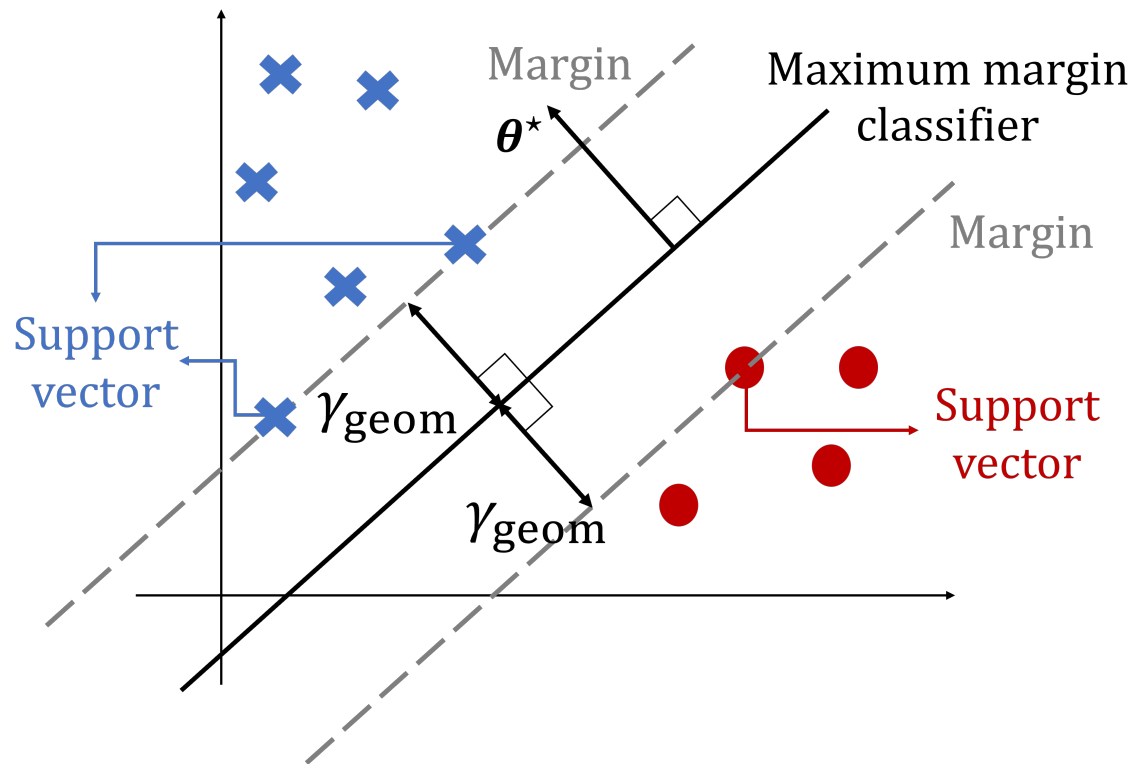
1. $\|\bar{\theta}\| < \|\theta_1\| = \|\theta_2\| \Rightarrow$ **Contradiction!**
2. $\|\bar{\theta}\| = \|\theta_1\| = \|\theta_2\| \Rightarrow$ Triangle inequality holds with equality. This can only happen if θ_1 is in the same direction as θ_2 .

In conclusion: there DO NOT exist 2 distinct min length solution vectors to PRIMAL-SVM.



Support Vector + Machine

- **Support vector** $\Leftarrow$ come from the input data set
influence the position of the max-margin classifier
- **Machine**: Input $\Rightarrow$ Machine $\Rightarrow$ Output classifier

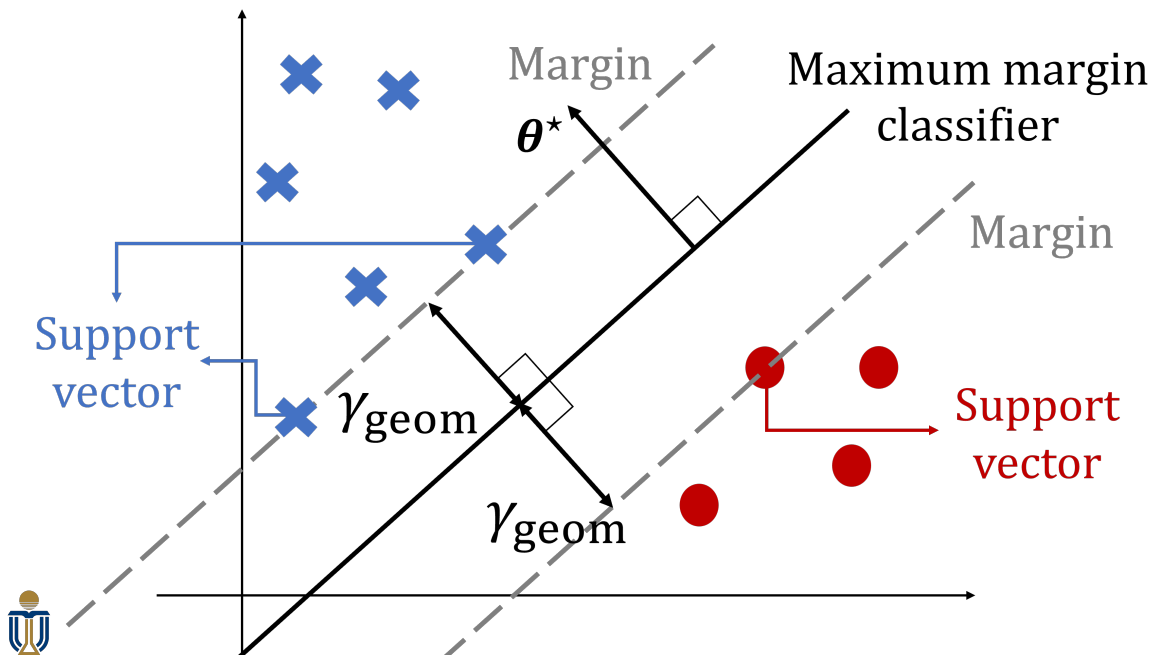


Robustness of max margin classifier

- As we have mentioned: larger margin $\Rightarrow$ more robust
- The (PRIMAL-SVM WITH OFFSET) problem:

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{1}{2} \|\boldsymbol{\theta}\|^2 \quad \text{s. t.} \quad y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t = 1, 2, \dots, n$$

- Solution depends on a small number of data samples $(\mathbf{x}_t, y_t)$



Support vectors:

- Samples exactly on the margin.
- Quantify the quality of this binary classifier.



Offset and slack

(PRIMAL-SVM WITH OFFSET AND SLACK)

$$\begin{aligned} \min_{\boldsymbol{\theta}, \theta_0, \boldsymbol{\xi} \in \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}_+^n} \quad & \frac{1}{2} \|\boldsymbol{\theta}'\|^2 + C \sum_{t=1}^n \xi_t \\ \text{s. t.} \quad & y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1 - \xi_t, \quad \xi_t \geq 0 \quad \forall t = 1, 2, \dots, n \end{aligned}$$

$\Longleftrightarrow$

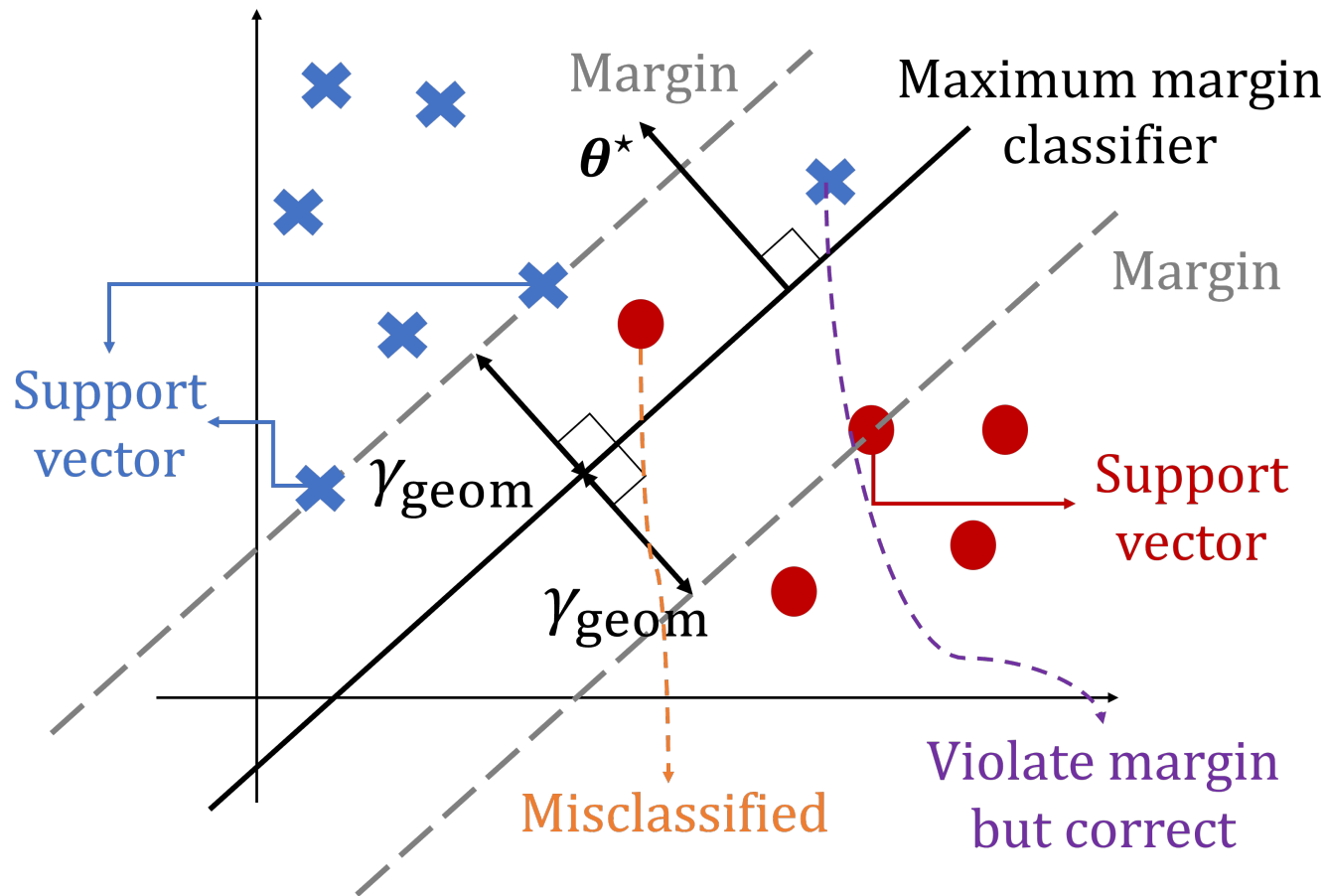
$$\min_{\boldsymbol{\theta}, \theta_0 \in \mathbb{R}^d \times \mathbb{R}} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 + C \sum_{t=1}^n \underbrace{\left[1 - y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0)\right]^+}_{:=\xi_t}$$

where $f(z) = (1 - z)^+ = \max\{0, 1 - z\}$.

- Why offset? Why slack?
- How did we come to this point?



Allowing misclassified example



Leave-one-out cross-validation error (LOOCV)

To assess the robustness of the SVM

$$\text{LOOCV} = \frac{1}{n} \sum_{t=1}^n \text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))),$$

- $(\boldsymbol{\theta}^{-t}, \theta_0^{-t})$ is learnt from the dataset with i^{th} sample left out, i.e., $\mathcal{D}^{-t} = \mathcal{D} \setminus \{\mathbf{x}_t, y_t\}$
- Loss function: $\text{Loss} = \mathbb{1}\{y \neq y'\}$ is zero-one loss

In other words,

$$(\boldsymbol{\theta}^{-t}, \theta_0^{-t}) \in \arg \min_{(\boldsymbol{\theta}, \theta) \in \mathbb{R}^d \times \mathbb{R}} \left\{ \frac{1}{2} \|\boldsymbol{\theta}\|^2 : y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t \in [n] \setminus \{t\} \right\}.$$



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)

Proposition 4.5

*Let the number of **support vectors** be $N \in \{0, 1, \dots, n\}$. Then*

$$\text{LOOCV} \leq \frac{N}{n}.$$

- If the number of support vectors is small, then LOOCV is also small and the SVM works well on new samples



Leave-one-out cross-validation (LOOCV) error

Hint for proof:

- If $\mathbf{x}_t$ is not a support vector, then it cannot influence the solution of the primal SVM.
- Hence,

$$f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, b^{-t})) = y_t, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

and

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) = 0, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

- For any other $\mathbf{x}_t \in \text{SV}$,

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) \leq 1, \quad \forall \mathbf{x}_t \in \text{Support Vectors (SV)}$$



Support Vector Machine (SVM)

- An algorithm that learns the **maximum (geometric) margin linear classifier**
- The solution is **unique**
- Incorporating an **offset** may result in larger geometric margin
- LOOCV to assess the **robustness** of SVM
- For non-linearly separable data: allow misclassified examples by incorporating a **slack**
- Solving SVM: Dual form, sequential minimal optimization (SMO)...
- ◉ Reading: Note of 6-867 (MIT)
Sec 7.2 and Chapter 12 of book “Mathematics for Machine Learning”



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering**
- 6 Model Selection + Feature selection



Recap of Lecture 5

- SVM (partially covered)
 - Linearly separable, γ -linearly separable
 - Margin, minimum margin, geometric margin
 - SVM: maximum margin linear classifier, unique solution
 - SVM: offset, slack, leave-one-out cross-validation error
- Feature engineering
 - Polynomial transformation, nonlinear transformation
 - Boolean, nominal, ordinal, missing values
- Model selection (partially covered)
 - Generalization of supervised learning algorithms
 - Error metric, regularizer, no free lunch!
 - Generalization, overfitting
 - Data partition: train, test and validation



Polynomial transformation

- Generalized Linear Discriminant Function

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{1 \leq i \leq j \leq d} w_{i,j} x_i x_j + \sum_{1 \leq i \leq j \leq k \leq d} w_{i,j,k} x_i x_j x_k$$

- Noting that $x_{l,i}$ is the i -th ($1 \leq i \leq d$) component of the l -th ($1 \leq l \leq m$) sample, we can stack this into

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{P}\mathbf{w} = \begin{bmatrix} \mathbf{p}_1^\top \mathbf{w} \\ \vdots \\ \mathbf{p}_m^\top \mathbf{w} \end{bmatrix} \quad \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \\ \vdots \\ w_{i,j} \\ \vdots \\ w_{i,j,k} \\ \vdots \end{bmatrix}$$

and

$$\mathbf{p}_l^\top \mathbf{w} = [1 \quad x_{l,1} \quad \dots \quad x_{l,d} \quad \dots \quad x_{l,i}x_{l,j} \quad \dots \quad x_{l,i}x_{l,j}x_{l,k} \quad \dots]$$



Notation: Boolean indicator function*

Define

$$\mathbb{1}(\text{statement}) = \begin{cases} 1 & \text{statement is true} \\ 0 & \text{statement is false} \end{cases}$$

Example:

- $\mathbb{1}(1 > 0) = 1$
- $\mathbb{1}(17 < 15) = 0$



Boolean variables*

- $\mathcal{X} = \{\text{true}, \text{false}\}$
- Transform to numerical variables:
let $\phi : \mathcal{X} \rightarrow \{0, 1\} : \phi(x) = \mathbb{1}(x)$.
- Example: bank customer dataset
 - $x_1 =$ with checking account?
 - $x_1 =$ with saving account?
 - $x_1 =$ with credit account?



Boolean expressions*

- $\mathcal{X} = \{\text{true}, \text{false}\}^2$
 $= \{(\text{true}, \text{true}), (\text{true}, \text{false}), (\text{false}, \text{true}), (\text{false}, \text{false})\}.$
- For $x = (x_1, x_2) \in \mathcal{X}$
- Let $\phi(x) = [\mathbb{1}(x_1), \mathbb{1}(x_2), \mathbb{1}(x_1 \text{ and } x_2), \mathbb{1}(x_1 \text{ or } x_2)]$
- Equivalent: polynomials in $[\mathbb{1}(x_1), \mathbb{1}(x_2)]$ span the same space
- Encodes logical expressions!



Nominal values: one-hot encoding*

- Nominal data: e.g., $\mathcal{X} = \{\text{apple}, \text{orange}, \text{banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}), \mathbb{1}(x = \text{orange}), \mathbb{1}(x = \text{banana})]$$

- One-hot encoding: only one element is non-zero
- Familiar?



Nominal values: one-hot encoding*

- **Nominal data:** e.g., $\mathcal{X} = \{\text{apple, orange, banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}), \mathbb{1}(x = \text{orange}), \mathbb{1}(x = \text{banana})]$$

- **One-hot encoding:** only one element is non-zero
- **Familiar?**

Recall multi-class classification with linear models

- **Extension:** sets,
e.g. you buy an apple and a banana at the same time



Ordinal values: real encoding/Boolean encoding

- Ordinal data: e.g., $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis

⊙ Real encoding:

$$\phi(x) = \begin{cases} 1, & x = \text{Stage I} \\ 2, & x = \text{Stage II} \\ 3, & x = \text{Stage III} \\ 4, & x = \text{Stage IV} \end{cases}$$

⊙ Boolean encoding:

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$



Missing values

- Remove rows/columns with missing entries
- (For time series) back-fill with most recent observed value
- Impute with mean, median, or mode
- Fancier imputation methods (unfortunately NOT covered in this class):
matrix completion, deep learning, ...
- Add new feature: Boolean indicator 1(data is missing)
 - Can detect if missingness is informative
 - Can complement imputation method
 - Can use different indicators for different kinds of missingness
(refused, missing, illegible response, ...)



Nonlinear transformations

- Sometimes data is easy to predict with a simple but nonlinear relation, e.g.

$$\log(y) = \mathbf{w}^\top \mathbf{x} \text{ equivalent to } y = \exp(\mathbf{w}^\top \mathbf{x})$$

can transform x or (even more important) y

- Familiar?
- Nonlinear transforms:
 - $\log$, $\exp$, quantile, median, ...
- More systematic ways to handle nonlinearities (unfortunately NOT covered here):
 - Deep learning, ...



Location

can be given as

- latitude, longitude
- zip code
- neighborhood, county, state, country

⊙ Can be transformed between these!

⊙ Which makes sense for your problem?

- Does nearness matter?
- Are there sharp boundaries? e.g., good district for this school but bad for another one
- Are other properties of the location (e.g., mean house price or crime rate) more important?



Outline

- 1 Intro of ML + mathematical tools
- 2 Linear regression (offset/multiple-output) + Linear classification + Polynomial regression
- 3 Ridge regression + Gradient descent (GD)
- 4 Logistic regression + Support vector machine (SVM)
- 5 Feature engineering
- 6 Model Selection + Feature selection**



Recap of Lecture 6

- Model selection
 - Generalization of supervised learning algorithms
 - Error metric, regularizer, no free lunch!
 - Generalization, overfitting
 - Data partition: train, test and validation
- Feature selection
 - Why? What?
 - Filter, wrapper, embedded



Generalization of supervised learning algorithms

- ⊙ We need the new ML system generalizes well on **unseen data**, e.g., data from my hospital or other hospitals
- ⊙ Elements in supervised learning:
 - Unknown target function $f : \mathcal{X} \rightarrow \mathcal{Y}$
 - Training examples $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$
 - Hypothesis set $\mathcal{H}$ (set of functions)
 - Learning algorithm $\mathcal{A}$
 - Final hypothesis $g : \mathcal{X} \rightarrow \mathcal{Y}$
- ⊙ How well will our classifier/regressor do on **new data**?
⇐ how well can g approximate f ?



Data partition: train, test and validation



- Fit model to the **training** dataset
- Fit hyperparameters to the **validation** dataset
- Test model performance on the **test** dataset



Error metric: some candidates

⊙ Regression

- Square error: $\text{error}_{\text{sq}}(y, y') = (y - y')^2$
- Absolute error: $\text{error}_{\text{abs}}(y, y') = |y - y'|$

⊙ Classification

- Misclassification error: $\text{error}_{\text{mis}}(y, y') = \mathbb{1}\{y \neq y'\}$
- Weighted misclassification error: If false positives are β times worse than false negatives

$$\text{error}_{\text{beta}}(y, y') = \beta \mathbb{1}\{y' = 1, y = -1\} + (1 - \beta) \mathbb{1}\{y' = -1, y = 1\}$$

- Balanced error rate: If dataset contains n_+ examples with positive labels and n_- negative

$$\text{error}_{\text{bal}}(y, y') = \frac{\mathbb{1}\{y \neq y'\}}{n_+ \mathbb{1}\{y\} + n_- \mathbb{1}\{1 - \mathbb{1}\{y\}\}}$$



Confusion matrix

		Predicted condition	
Total population = P+N		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)
		Precision: $\frac{TP}{TP + FP}$	Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$



Some ways to further evaluate classification model

- **Precision** = $\# \text{ true pos} / (\# \text{ true pos} + \# \text{ false pos})$
 - Measure the accuracy of positive predictions
- **Recall (sensitivity)** = $\# \text{ true pos} / (\# \text{ true pos} + \# \text{ false neg})$
 - Measures the model's ability to find all the positive instances
- **F1 score** = $2 / (\text{Recall}^{-1} + \text{Precision}^{-1})$
 - Harmonic mean of precision and recall: balance the two metrics into a single number, making it especially useful when precision and recall are in trade-off



Some ways to further evaluate classification model

- True Positive Rate = $\# \text{ true pos} / (\# \text{ true pos} + \# \text{ false neg})$
 - Measure the accuracy of prediction for positive samples
- False Positive Rate = $\# \text{ false pos} / (\# \text{ false pos} + \# \text{ true neg})$
 - Measure the accuracy of prediction for negative samples
- ROC, AUC (AUROC)
 - ROC: Receiver Operating Characteristic
 - AUC (AUROC): Area Under the Receiver Operating Characteristic
 - Remove dependence on thresholds



Some ways to further evaluate classification model

- Multiclass?

- Calculate precision, recall within each class
- Average them across the classes
- Consider balanced average?

- Python:

- `sklearn.metrics.average_precision_score`
- `sklearn.metrics.recall_score`
- `sklearn.metrics.precision_score`
- `sklearn.metrics.f1_score`
- `sklearn.metrics.roc_curve`
- `sklearn.metrics.auc`



Train and test error: evaluate the selected error metric on dataset

- Partition data $\mathcal{D}$ into training set $\mathcal{D}_{\text{train}}$ and test set $\mathcal{D}_{\text{test}}$
 $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset, \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}$
- Algorithm $\mathcal{A}$ is **only** allowed to see the training set $\mathcal{D}_{\text{train}}$

- **Training error**

$$E_{\text{train}}(g) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \text{error}(y_i, g(x_i))$$

Test error

$$E_{\text{test}}(g) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} \text{error}(y_i, g(x_i))$$

- Mean Error: “Mean Square Error” (MSE), “Mean Absolute Error” (MAE), etc.



Objective function = Error + Regularizer



Regularization

⊙ Why regularize?

- Reduce **variance** of the model (more stable)
- Prevent overfitting
- Impose prior structural knowledge
- Improve interpretability: simpler model, automatic feature selection

⊙ Why NOT regularize?

- Gauss-Markov theorem:
 - Least squares is the best linear **unbiased** estimator
- Regularization increases **bias**



No free lunch! Trade-off between bias and variance!

Decompose the expected error:

$$\text{expected prediction error} = \text{bias}^2 + \text{variance} + \text{noise}$$

- **bias²**: average prediction error over all data sets (squared)
- **variance**: how much solutions for different data sets vary (stability)
- **noise**: deviation of measurements from the true value (unavoidable error)

Theorem 6.1 (No free lunch!)

Problem: *Low bias comes with high variance and low variances comes with high bias!*

- **Too high bias**: *we cannot fit the data well, because our solutions are too restricted.*
 - **Too low bias**: *we have too high variance, sensitive to data changes (overfitting).*
- *We need to find the **sweet spot** inbetween!*



Regularizer

We might choose regularizer so models will be

- small
- sparse
- nonnegative
- smooth
- ...
- ℓ_1 -norm or ℓ_2 -norm? Lasso?



ℓ_p -norm of vector

Definition 6.2 (ℓ_p -norm of vector)

Let $p \in (0, \infty)$, ℓ_p -norm of $\mathbf{w} \in \mathbb{R}^d$ is defined as $\|\mathbf{w}\|_p = \left(\sum_{i=1}^d |w_i|^p \right)^{1/p}$.

- $0 < p < \infty$:

- ▶ ℓ_1 -norm is $\|\mathbf{w}\|_1 = \sum_{i=1}^d |w_i|$. ℓ_2 -norm is $\|\mathbf{w}\|_2 = \sqrt{\sum_{i=1}^d |w_i|^2}$.

- $p = 0$ or ∞ , ℓ_p -norm is defined by taking limit:

- ▶ ℓ_∞ -norm is $\|\mathbf{w}\|_\infty = \lim_{p \rightarrow \infty} \left(\sum_{i=1}^d |w_i|^p \right)^{1/p} = \max_i |w_i|$.

- ▶ ℓ_0 -norm is $\|\mathbf{w}\|_0 = \lim_{p \rightarrow 0} \left(\sum_{i=1}^d |w_i|^p \right)^{1/p} = \sum_{i=1}^d \mathbb{1}(w_i \neq 0)$.

- Technical note: ℓ_0 is not actually a norm

(not absolutely homogeneous since $\|\alpha \mathbf{w}\|_0 = \|\mathbf{w}\|_0$ for $\alpha \neq 0$)



Generalization and overfitting

- Goal of model is **not** to predict only well on $\mathcal{D}$
- Goal of model is to predict well on **all data (old, new and unseen)**
- If the model has ?? training set error and ?? test set error, we say the model:

	Low test set error	High test set error
Low training set error	Generalize	Overfit
High training set error	?!?!	Underfit



How to choose train and test tests

- At random (e.g., toss random coin with probability p)
- More data in the training set improves the model fit
- More data in test set helps determine how well the model will work out of sample
- Rule of thumb: put about 20% of data into the **test set**
- ...

(https://scikit-learn.org/stable/api/sklearn.model_selection.html)



Validation

⊙ A simple and effective validation procedure:

1. Split data into training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{valid}}$

2. Pick m different interesting model classes

e.g., different model class ϕ : $\phi_1, \phi_2, \dots, \phi_m$

3. Fit (“train”) models on training set $\mathcal{D}_{\text{train}}$

Get one hypothesis $h : \mathcal{X} \rightarrow \mathcal{Y}$ for each ϕ s, and set

$$\mathcal{H} = \{h_1, h_2, \dots, h_m\}$$

4. Compute error of each hypothesis on validation set $\mathcal{D}_{\text{valid}}$ and choose the lowest:

$$h^* = \arg \min_{h \in \mathcal{H}} E_{\mathcal{D}_{\text{valid}}}(h)$$

○ OR: select the model class according to h^* and retrain h^* on the entire training set.



Cross-validation

1. Pick a bunch of interesting model classes (e.g., different ϕ s: $\phi_1, \phi_2, \dots, \phi_m$)
2. For each possible split of data into training set $\mathcal{D}$ and validation set $\mathcal{D}'$:

$$(\mathcal{D}_1, \mathcal{D}'_1), (\mathcal{D}_2, \mathcal{D}'_2), \dots, (\mathcal{D}_K, \mathcal{D}'_K)$$

- Fit ('train') each model (class) ϕ on training set $\mathcal{D}$ and obtain hypothesis $h_{\phi, \mathcal{D}}$
 - Compute error of each hypothesis on validation set $\mathcal{D}'$: $E_{\mathcal{D}'}(h_{\phi, \mathcal{D}})$
 - Now, the validation set $\mathcal{D}'$ is not a **single** set
 - $\Rightarrow$ multiple validation sets $\mathcal{D}'$ s
3. Estimate error of model as **average** of error on each $\mathcal{D}'$

$$\phi^* = \arg \min_{\phi_i} \frac{1}{K} \sum_{j=1}^K E_{\mathcal{D}'_j}(h_{\phi_i, \mathcal{D}_j})$$

4. Fit the model ϕ^* using the entire dataset.



Cross-validation: how to pick splits

- **Leave-one-out cross-validation**

- $\mathcal{D}'$ has one example, $\mathcal{D}$ has all the rest
 - ▶ Advantage: accurate
 - ▶ Disadvantage: slow
 - ▶ Recall: measure the robustness of SVM

- **n -fold cross-validation**

- $\mathcal{D}'$ has $1/n$ of the entire training dataset, $\mathcal{D}$ has all the rest (usually, $n = 5$ or 10)
 - ▶ Decently accurate, not too slow



Go further

Exercise:

- `sklearn.metrics`
- `sklearn.model_selection`

References:

- Sections 8-9 in Note of CS299 (Stanford)
- Webpage: (online note)
- Book: Bishop, C.M. 2007. Pattern recognition and machine learning, 5th edition. Springer.



Types of feature selection

- **Filter:** Rank features (e.g., correlation) and abandon a subset of features before modeling.
- **Wrapper:** Test feature subsets with a model (e.g., sequentially add/drop features).
- **Embedded:** Selection during training (e.g., feature importance).
 - Bagging and boosting often use embedded methods!



Good luck in your midterm!

